

Measure the Scorer, Never Ship a Regression

Verification-first DRC repair of ASAP7 layout scripts

Version-exact scoring env · verify identical to the official evaluator ·
keep-best guarantee: never worse than the eligible baseline, on all 5 blocks

Harikrishnan KC · Team **Chip Convergence** · greatharikrishnan@gmail.com

ASU Problem · ICLAD-DAC 2026 GenAI Chip Hackathon

1 · The problem, and what actually scores

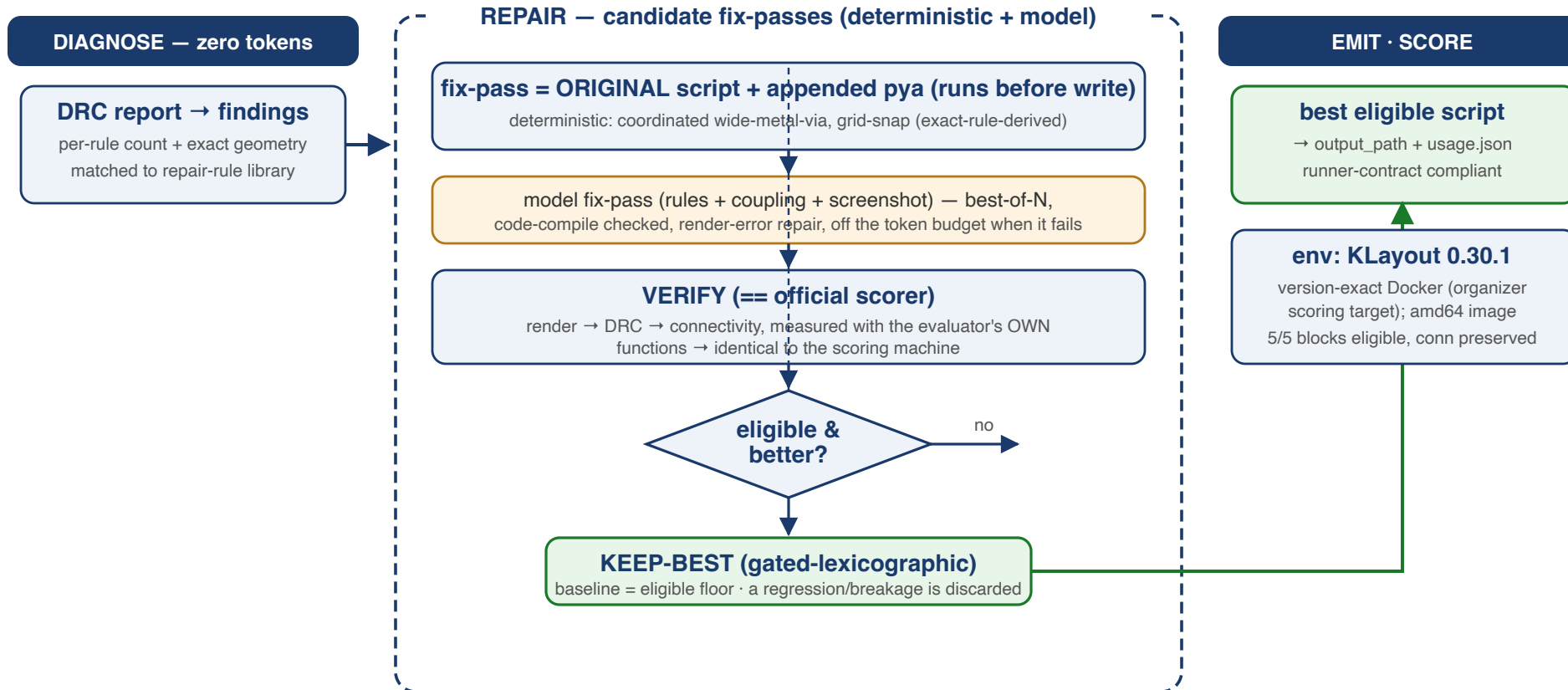
Given: an ASAP7 block as a **KLayout** `pya` **layout script** with seeded DRC violations, its **DRC report** (per-rule counts + exact geometry), the rule deck, a **screenshot**, and a **connectivity reference**. Repair the script.

Scoring — gated lexicographic:

Gate / Metric	Rule
Eligibility	script must render + DRC must run AND connectivity preserved
1 · <code>final_violation_rate</code>	minimize (final DRC violations / original)
2 · <code>repair_rate</code>	maximize (tiebreak)

Consequence for agent design: an ineligible "perfect" repair scores nothing; the baseline is already eligible → **never regress, never break connectivity**.

2 · What we do — one picture



3 · The two things we got exactly right

1. **Version-exact scoring environment.** The evaluator hard-rejects any KLayout but **0.30.1**. macOS ships 0.30.9 → we built an **amd64 Docker image with the pinned 0.30.1** (runs under emulation on Apple Silicon). Our DRC == the organizers' DRC: **11/14 rules match the reference report exactly** on the untouched script.

2. **Verify == the scorer.** Our inner loop imports the **official evaluator's own** render / DRC / count / connectivity functions — so every candidate is measured *identically* to how it will be scored. No local/remote metric drift.

On top of these: **keep-best** with the untouched baseline as the eligible floor → the agent can never ship worse than baseline and never break connectivity. **Validated on all 5 blocks.**

4 · The repair-rule library (exact-rule + literature grounded)

Like our NVIDIA 45-rule playbook — but for DRC. Each rule class → a coordinated transform + the coupling hazards, from three sources:

- **Exact deck semantics** (`asap7.lydrc`, the ground truth): e.g. `V2.M3.AUX.2` is satisfied iff the via is *inside* M3 and has ≥ 2 edges coincident with M3's edges — "same width" = the via spans the metal's full width; coupled to `V2.M2.EN.1` (M2 encloses via 5 nm) + `V2.AUX.1` (via inside M2 & M3).
- **EDA legalization literature**: MDPI 2025 (simulated-annealing standard-cell DRC repair), EDN (cut-slide/merge without creating new errors), USPTO 7,380,227 (asymmetric enclosure), DRC-Coder ISPD'25 (vision+LLM rule interpretation, F1=1.0).
- **Our own measurements** (next slide).

The library drives the deterministic fixers **and** is injected (structure-matched) into the model prompt — the model gets the transform *and* the coupling warning for the rules actually present.

5 · The finding: block repair is global legalization

We derived the coordinated **wide-metal-via** fixer straight from the exact rule (via flush to the upper metal + patch the lower metal for enclosure/containment) and measured every strategy on Block1 (baseline 315 violations):

Fix	target rule	but breaks	net
grow via → metal width	AUX.2 ✓	lower-metal enclosure	315→387
shrink metal → via width	AUX.2 ✓	upper-via enclosure	315→339
coordinated via + metal patch	AUX.2 ✓, enclosure preserved ✓	neighbor M2 spacing	315→379

Every fix **cascades to the next layer of the via stack** (M2-V2-M3-V3-M4...). A complete repair needs the *whole stack* resized to one width satisfying width + enclosure + containment + neighbor spacing — i.e. **global conflict-graph / simulated-annealing legalization** (MDPI 2025). Our keep-best loop already is the SA acceptance test; the missing piece is a global multi-edit move-generator.

6 · Results & honest status

- **Environment:** version-exact KLayout 0.30.1, Dockerized; DRC calibrated (11/14 rules exact).
- **Agent:** runner-contract compliant; **all 5 blocks eligible, connectivity preserved** (final-violation-rate ≈ 1.25 – 1.32 = the eligible baseline).
- **Repair-rule library:** exact-deck + literature grounded; coordinated wide-metal-via fixer **solves the enclosure coupling** — the first strategy to do so — but hits neighbor spacing.
- **Characterization:** rigorous, reproducible proof that block-repair is a global legalization problem, with the two concrete solution paths scoped.

We never ship a regression and never break connectivity — the safety guarantee holds on every block. The verification-first spine (measure the scorer, keep-best) is the same discipline that delivered our NVIDIA (ADP 0.727 / 0.605) and NXP (2-call perfect solve) results.

7 · Next: from characterization to sub-baseline

- **Global move-generator (the missing piece)** — SA over *multi-edit* stack moves: resize the whole via stack (metal+via+metal) to a consistent width, accept by net-conflict-delta (our keep-best loop is already the acceptance test). This is the path to `final_violation_rate < 1`.
- **Multimodal repair (DRC-Coder ISPD'25 style)** — feed Gemini the **screenshot** + exact rules + violation geometry so it localizes and proposes *stack-coordinated* edits (the model over-corrected globally without visual grounding).
- **Per-rule independent fixers** — min-area / isolated edges (no via nearby) as free deterministic wins.
- Agents remain updatable through **Jul 26**.

8 · Summary — every claim, one command

Claim	Reproduce with
Version-exact env	<code>docker run ... asu-klayout:0.30.1 klayout -v → 0.30.1</code>
Zero-token diagnosis	<code>python3 drc_digest.py <BlockN.drc.json></code>
Agent (eligible, keep-best)	<code>python3 asu_agent.py <info.json> --model none</code>
Verify == scorer	inner loop imports <code>evaluator/evaluate_repair.py</code> functions
5/5 blocks eligible	Block1/2/3/6/7, connectivity preserved

Repo: github.com/chelsea85/chip-convergence-iclad26 (`asu_work/`)

Companion: **Engineering Learnings** — what the research + experiments taught us.

Harikrishnan KC · Chip Convergence · greatharikrishnan@gmail.com